

Team	Team 1 - Topic D2 - Booking platform
Member	1. Chử Vũ Thảo Hiền - 11247287 2. Phan Thị Anh Quỳnh - 11247347 3. Bùi Phương Thảo - 11247353
Github	https://github.com/SE-FDA-NEU-AI66B/software-group1-topicd2.git
Pull Request	Add docs/process.md dossier by dqchien · Pull Request #5 · SE-FDA-NEU-AI66B/software-group1-topicd2
Merge commit	Process.md submitted by: Chử Vũ Thảo Hiền

Section 1 — Chosen process and its position on the spectrum

(a) Process model: Hybrid — "Plan-driven Foundation + Agile Incremental Sprints"

Phase 0 — Foundation (done once, fixed for the whole term): The team jointly analyzes requirements, designs the overall architecture, and designs the database schema for the booking table (a unique constraint on time slot + resource, and a transaction mechanism to prevent double booking). Since this is the most stable part of the requirements and also the project's biggest technical risk, the team builds a small prototype to verify the conflict-prevention mechanism before finalizing the design.

One cycle (internal sprint, several sprints occur between two milestones):

Steps	Who does	Content	Output
1. Sprint Planning	Whole team	Select stories from the backlog, open corresponding GitHub Issues, assign tasks by module	Sprint backlog on GitHub Issues
2. Detailed design	Module owner	Design the booking flow/interface/API for the feature in the sprint	Module design document

3. Implementation	Each member	Code on a separate branch, open a Pull Request upon completion	Pull Request, source code
4. Code review & Testing	Whole team	Review each other's PRs before merging; test specifically for the anti-double-booking feature (simulating concurrent requests)	Merged PR, test report
5. Internal review	Whole team	Consolidate progress, prepare for the nearest milestone	Incremental build
6. Retrospective	Whole team	Draw lessons learned, adjust the backlog	Updated backlog

At each milestone (4 fixed occurrences) + the final demo: the team presents the current version to the instructor (the sole customer), receives formal feedback, and uses that feedback to adjust backlog priorities for the following sprints.

(b) Position on the spectrum — revised version

Positioning: ~30% Plan-driven + 70% Agile.

Fixed for the whole term:

- Main purpose/functionality: allowing users to book available time slots and preventing two people from booking the same slot — as this is the most stable part of the requirements and does not change
- System architecture, database schema, and the anti-double-booking rule — changing this midway carries high risk, so it must be finalized early
- 4 milestones and the final demo date — fixed due to course constraints, cannot be moved

Flexible by cycle:

- The specific booking flow, interface layout, reminder function, usage frequency statistics — can change based on the instructor's feedback at each milestone
- Backlog priority order and task assignment within the team each sprint

Reasons for the choice:

- The project has a low safety/legal impact (a double-booking error only causes inconvenience, with no serious consequences), so the team does not need a strict change-control process like Spiral — only the anti-double-booking rule needs to be carefully documented, since all members must follow the same standard.
- The team is small (3 people, with low communication cost thanks to GitHub Issues/PRs) and the customer only provides feedback at fixed milestones, so an Agile-leaning Hybrid is a better-founded choice than Waterfall (too rigid, with only a single delivery) or Spiral (heavy on risk analysis that is unnecessary here).

Section 2 — The five diagnostic questions

1. Are your requirements stable or volatile?

Our requirements are mostly stable. We already know that the main purpose of the system is to let users book available time slots and prevent two users from booking the same slot. These core functions are unlikely to change. However, smaller parts such as the booking steps, screen layout, reminder feature, and usage statistics may be changed if the instructor gives feedback or if the team finds problems while building the system. Therefore, we expect changes mainly in the details, not in the main purpose of the project.

2. Does the project carry safety or legal impact that would demand formal documentation and change control?

Our project has low safety and legal impact. The system is used to book rooms, sports fields, or services. If something goes wrong, such as two users trying to book the same time slot, the main result is inconvenience or a wrong booking. It does not involve serious physical safety risks or major legal consequences. Therefore, we do not need a complicated process for documenting and approving every change. However, important changes, such as changing the rule that prevents double booking, should still be recorded so that all team members follow the same rule.

3. Is your team large and distributed, or small and co-located? How does that affect communication cost?

Our team is small, so communication is relatively easy. Members can quickly discuss problems, divide tasks, and ask for help without going through many people. This keeps the communication cost low. However, team members may work on different parts of the booking system at the same time. If decisions are only discussed in chat, some information may be forgotten. Therefore, we will use GitHub Issues and Pull Requests to record tasks, decisions, and code changes so that everyone can see what is happening.

4. Can your customer engage continuously, or only at fixed checkpoints? Our main customer is the course instructor, and feedback is mainly available at the four fixed milestones and the final demo. The instructor is not available to check every small feature

during development. Therefore, the team has to make decisions and continue working between the checkpoints. At each milestone, we can show the current system, receive feedback, and use that feedback to improve the next version.

5. What do organizational culture and contract constraints allow?

The project has four fixed milestones and a fixed final demo date because it is a semester course project. These deadlines cannot be changed, so the team must decide which features need to be completed before each milestone. However, we can still change the order of tasks and improve features during the semester. For example, we can build the basic booking function first and add reminders or usage statistics later. Therefore, our deadlines are fixed, but the work between those deadlines can remain flexible.

Section 3 — Critical thinking: risks of the opposite choice

- If our team moved from the proposed hybrid process to a fully plan-driven process, the biggest risk would be losing the ability to adapt the less-stable parts of the booking platform after milestone feedback.
- Our team intentionally keeps the booking flow, user interface, reminder features, usage statistics, and backlog priorities flexible so they can be refined based on instructor feedback. Under a fully plan-driven process, these decisions would be fixed much earlier, making later changes more costly and increasing rework.
- The first concrete symptom would be that the implemented booking flow or interface does not match the instructor's latest feedback, while the team has to continue following the original plan.

Section 4 — Process rules our team commits to

1. Sprint length and backlog: Each internal sprint lasts two weeks. At the beginning of each sprint, the team reviews and re-prioritizes the backlog in GitHub Issues.

2. Pull Request review: Every code change must be developed on a separate branch, submitted through a Pull Request, and reviewed by at least one other team member before being merged into main.

3. Requirement changes: Any change to flexible requirements, such as the booking flow, interface, reminders, or usage statistics, must be recorded in docs/changelog.md and reflected in the backlog before implementation.

4. Anti-double-booking verification: Every change affecting booking creation, modification, or cancellation must include a test verifying that concurrent requests cannot successfully create two bookings for the same resource and time slot.

5. Foundation and milestones: The agreed system architecture, database schema, and core anti-double-booking rule are reviewed and approved before implementation of later modules.